Complete Guide to `pd.read_json()` & JSON Handling in Pandas

---

1. What is JSON & Why Pandas Uses It

JSON (JavaScript Object Notation) is the most common data-exchange format used in APIs, web apps, config files, logs, and NoSQL databases. Unlike CSV, JSON supports **nested objects**, **arrays inside rows**, and **mixed datatypes**.

> 💡 **Core Idea**
>
> `pd.read_json()` converts JSON → structured parsing → column inference → index creation → dtype detection → **DataFrame**
>
> .

2. Basic Usage

2.1 Read from a JSON File

```
import pandas as pd

df = pd.read_json("file.json")
print(df)
```

```
    name  age
0  Sahil   24
1   Riya   22
```

2.2 Read from a Python Dictionary

```
import pandas as pd

data = {
    "Duration": {"0":60, "1":60, "2":45},
    "Pulse":    {"0":110,"1":117,"2":109}
}
df = pd.DataFrame(data)
print(df)
```

```
   Duration  Pulse
0        60    110
1        60    117
2        45    109
```

2.3 Read from a JSON String (using StringIO)

```
import pandas as pd
from io import StringIO

json_data = '[{"A":1,"B":2}, {"A":3,"B":4}]'

df = pd.read_json(StringIO(json_data))
print(df)
```

```
   A  B
0  1  2
1  3  4
```

2.4 Read from a URL (API)

```
import pandas as pd

url = "https://jsonplaceholder.typicode.com/posts/1/comments"
df = pd.read_json(url)
print(df.head())
```

3. Function Signature

```
pd.read_json(
    path_or_buf,          # File path, JSON string, URL, or file object
    orient=None,          # How JSON maps to rows/columns
    typ='frame',          # Return type: 'frame' or 'series'
    dtype=None,           # Force specific datatypes
    convert_dates=True,   # Auto-detect date columns
    keep_default_dates=True,
    precise_float=False,  # Higher float precision
    lines=False,          # One JSON object per line
    chunksize=None,       # Read in chunks (big data)
    compression='infer'   # Handle gzip, bz2, etc.
)
```

4. The `orient` Parameter (Most Important)

This tells Pandas **how the JSON structure maps to rows and columns**.

orient	JSON Shape	When to Use
`"records"`	`[{col:val, …}, …]`	Most common — APIs return this
`"index"`	`{row_label: {col:val}}`	Outer keys → row index
`"columns"`	`{col: {row:val}}`	Column-oriented data (default)
`"values"`	`[[val, val], …]`	Raw matrix, no labels
`"split"`	`{columns:[], index:[], data:[]}`	Efficient system export/import
`"table"`	`{schema:{…}, data:[…]}`	Production — includes schema

4.1 orient = "records" (Most Common)

```
df = pd.read_json("file.json", orient="records")
# Each JSON object = one row.  APIs usually return this format.
```

4.2 orient = "index"

```
# JSON: {"row1": {"A":10,"B":20}, "row2": {"A":30,"B":40}}
df = pd.read_json("file1.json", orient="index")
```

```
       A   B
row1  10  20
row2  30  40
```

4.3 orient = "columns"

```
# JSON: {"A": {"0":1,"1":3}, "B": {"0":2,"1":4}}
df = pd.read_json("file2.json", orient="columns")
```

```
   A  B
0  1  2
1  3  4
```

4.4 orient = "values" (No Labels)

```
df = pd.read_json("file3.json", orient="values")
df.columns = ["A", "B"]   # Assign column names manually
```

```
   A  B
0  1  2
1  3  4
```

4.5 orient = "table" (Schema + Data)

```
# Step 1 — Export with table orient
json_data = df.to_json(orient='table')

# Step 2 — Read back
new_df = pd.read_json(StringIO(json_data), orient='table')
```

> ✅ **Why "table" is powerful**
>
> It preserves column types, index info, and schema — ideal for production data pipelines.

5. Other Key Parameters

5.1 `typ` — Return Type

```
# Returns a DataFrame (default)
df = pd.read_json("file.json", typ="frame")

# Returns a Series
s = pd.read_json("file.json", typ="series")
```

5.2 `dtype` — Force Datatypes

```
# Force "age" to float instead of auto-inferred int
df = pd.read_json("file.json", dtype={"age": float})
```

```
    name   age
0  Sahil  24.0
1   Riya  22.0
```

> 💡 **Why use dtype?**
>
> JSON inference may pick wrong types. Production pipelines require a fixed, predictable schema.

5.3 `convert_dates` — Auto Date Conversion

```
df = pd.read_json("date.json", convert_dates=["date"])
```

Pandas auto-detects columns named `date`, `datetime`, `timestamp` and converts them to `datetime64`.

5.4 `keep_default_dates`

```
# True (default): auto-convert date-like column names
df = pd.read_json("file.json", keep_default_dates=True)
print(df.dtypes)
# date    datetime64[ns]
```

5.5 `precise_float` — Float Precision

```
# Higher precision (slower)
df = pd.read_json("file.json", precise_float=True)
# salary → 12345.578901

# Default (faster, may lose precision)
df = pd.read_json("file.json", precise_float=False)
```

5.6 `lines=True` — Line-Delimited JSON

Very common in **logs and big data**. Each line is a separate JSON object.

```
# file5.json contains:
# {"name":"A","age":20}
# {"name":"B","age":25}
# {"name":"C","age":30}

df = pd.read_json("file5.json", lines=True)
```

```
  name  age
0    A   20
1    B   25
2    C   30
```

> ⚠️ **Without `lines=True`**
> You'll get `ValueError: Trailing data` because Pandas tries to parse the whole file as one JSON object.

5.7 `chunksize` — Big Data Handling

```
# Read 2 rows at a time (returns iterator, not DataFrame)
reader = pd.read_json("bigfile.json", lines=True, chunksize=2)

for chunk in reader:
    print(chunk)
```

5.8 `compression` — Compressed Files

```
df = pd.read_json("data.json.gz", compression="gzip")
```

6. Handling Nested JSON (Very Important)

Real-world APIs almost always return nested JSON. `read_json()` keeps nested dicts as `object` type.

Solution: `pd.json_normalize()`

```
import pandas as pd

data = [{
    "name": "Sahil",
    "marks": {"math": 90, "science": 95}
}]

df = pd.json_normalize(data)
print(df)
```

```
    name  marks.math  marks.science
0  Sahil          90             95
```

> ✅ **Key Takeaway**
>
> `json_normalize()` flattens nested dicts into dot-separated columns — essential for working with API data.

7. `json.load()` vs `pd.read_json()`

Feature	`json.load()`	`pd.read_json()`
Returns	Python dict / list	DataFrame (analysis-ready)
Date handling	Manual	Automatic
Big data support	No	Yes (chunksize, lines)
Orient control	No	Yes (records, index, etc.)
Best for	Raw data manipulation	Data analysis & pipelines

8. Real-Life Example

```
import pandas as pd
from io import StringIO

orders = '''
[
  {"order_id":1, "customer":"Sahil", "amount":500},
  {"order_id":2, "customer":"Riya",  "amount":700},
  {"order_id":3, "customer":"Aman",  "amount":400}
]
'''

df = pd.read_json(StringIO(orders))
print(df)
print("Total:", df["amount"].sum())     # 1600
print("Average:", df["amount"].mean())  # 533.33
```

```
   order_id customer  amount
0         1    Sahil     500
1         2     Riya     700
2         3     Aman     400
Total: 1600
Average: 533.3333333333334
```

9. Quick Reference Cheat Sheet

Task	Code
Read JSON file	`pd.read_json("file.json")`
Read from URL	`pd.read_json("https://api.example.com/data")`
Read JSON string	`pd.read_json(StringIO(json_str))`
Read API-style JSON	`pd.read_json("f.json", orient="records")`
Read line-delimited	`pd.read_json("f.json", lines=True)`
Force column types	`pd.read_json("f.json", dtype={"age": float})`
Flatten nested JSON	`pd.json_normalize(data)`
Read in chunks	`pd.read_json("f.json", lines=True, chunksize=100)`
Read compressed	`pd.read_json("f.json.gz", compression="gzip")`